

Multi-layer perceptrons

1 Introduction

In the previous [chapter](#) we learned about the basics of neural networks, and discussed briefly a particular type of neural network, the multi-layer perceptron. In this chapter, we are going to learn more about this particular model, while getting some more details on the ingredients and how to run neural networks.

As described in the previous chapter, a [multilayer perceptron \(MLP\)](#) (*read linked content*) is a feedforward fully connected neural network. It is **feedforward**, because, as we have already heard, the computation passes the input values toward the output in a forward direction creating an acyclic graph (e.g. with no cycles or backward links). It is **fully-connected** because each node after the input layer receives input from every node in the previous layer, making the dense connectivity (e.g. full pairwise connection) shown in [Figure 1](#) below. The hierarchical structure of the network implies that the graphics represent a nested function.

An MLP usually consists of 3 or more layers: an input layer, an output layer and one or more hidden layers also called **dense** or **FC layers**. We know the values of the nodes in the first layer (e.g. our input), and the number of output neurons will be equal to the number of classes in the classification problem. All others are hidden neurons, and the layers that contain them are called **hidden layers**.

Sections [5.2](#) and [5.3](#) (*read these*) in Dive into Deep Learning explains the implementation of MLP and inner workings of computational graphs and relevant algorithms.

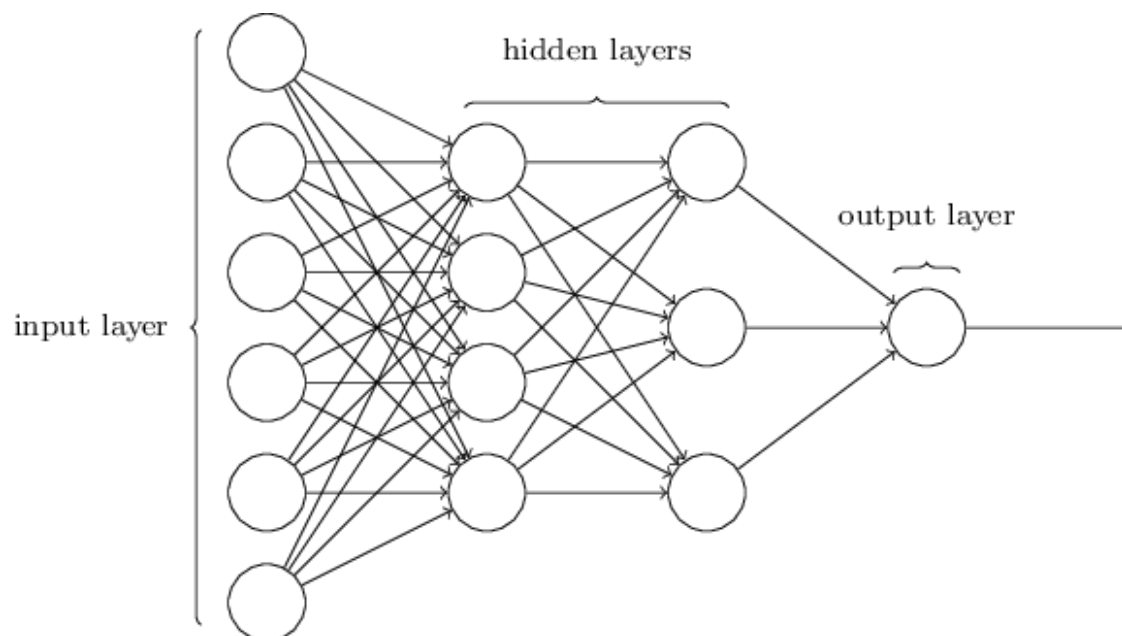


Figure 1: Example architecture of a MLP with two hidden layers and a single output node ([credit](#))

Note

As discussed in the [last chapter](#), we generally call a neural net with a single hidden layer a shallow neural network, and refer to a network with two or more hidden layers as a deep neural network. However, this terminology is not universal, and sometimes you will see the words “shallow” and “deep” used subjectively to denote networks with a “few” and with “many” layers, respectively.

Beside having more hidden layers, modern deep neural networks have other improvements, such as better activation functions (e.g. ReLU family), and usually include batch normalization and dropout layers in their design.

We described previously how each computation unit in a FC layer accepts the input, multiplies weights, adds the bias, and applies an activation function to it before forwarding it to the next layer. This activated output becomes input for the computation units in the next layer.

Refreshing ourselves on the formulas seen before (but presenting them slightly differently), each unit in a dense layer therefore performs the computations as shown below, the first of which involves an affine transformation of the input, while the second computation is to apply a point-wise non-linearity to the output of the first stage.

$$\begin{aligned} a_1 &= W_1 I + b_1 \\ z_1 &= \sigma_1(a_1) \end{aligned} \tag{1}$$

Where σ_1 is the non-linear function in the first layer, W_1 is the weight matrix represented by the connections between the input layer I (with 6 features, or nodes) in [Figure 1](#), and b_0 is the bias vector. Note that [Equation 1](#) is a two-stage equivalent of [Equation 6](#) in the previous chapter.

For the next layer (e.g. in our example its the second hidden layer), the calculation is as follows:

$$\begin{aligned} a_2 &= W_2 z_1 + b_2 \\ z_2 &= \sigma_2(a_2) \end{aligned}$$

A third pair of these equations would connect the second layer and the output. A general form of the these can be written as:

$$z_l = \sigma_l(z_{l-1}) = \sigma_l(b_l + W_l(z_{l-1}))$$

2 Commonly used activation functions

Up to now, we have mentioned activation functions, and the fact that they introduce non-linearity into the networks, but haven't really given any detail, so we will look at a few of the more widely used ones (and you should familiarize yourself with the relevant [section](#) (*read this*) in Dive into Deep Learning.

The activation function in a neural network has a huge impact on the model's final performance, thus the choice of activation function used in the network is crucial.

We'll first show two exponentiated functions, sigmoid and tanh, which were among the first used (and are still used in certain kinds of networks), but are now largely replaced by the simpler and less computationally intensive ReLu (and its derivatives). The reason for their replacement is many deep neural networks that use these two functions, and which are trained using backpropagation, suffer from the problem of [vanishing gradients](#) (*read this*).

2.1 Sigmoid (logistic) function

The sigmoid, or logistic, function ([Figure 2](#)) was one of the first used in neural networks. It maps real numbers into the unit interval (0, 1), which is useful if you want to transform a real-valued variable into something that represents a probability. This function is used as an activation, and also can be used at the end of the network in order to classify binary variables (for multi-class mappings, softmax is used).

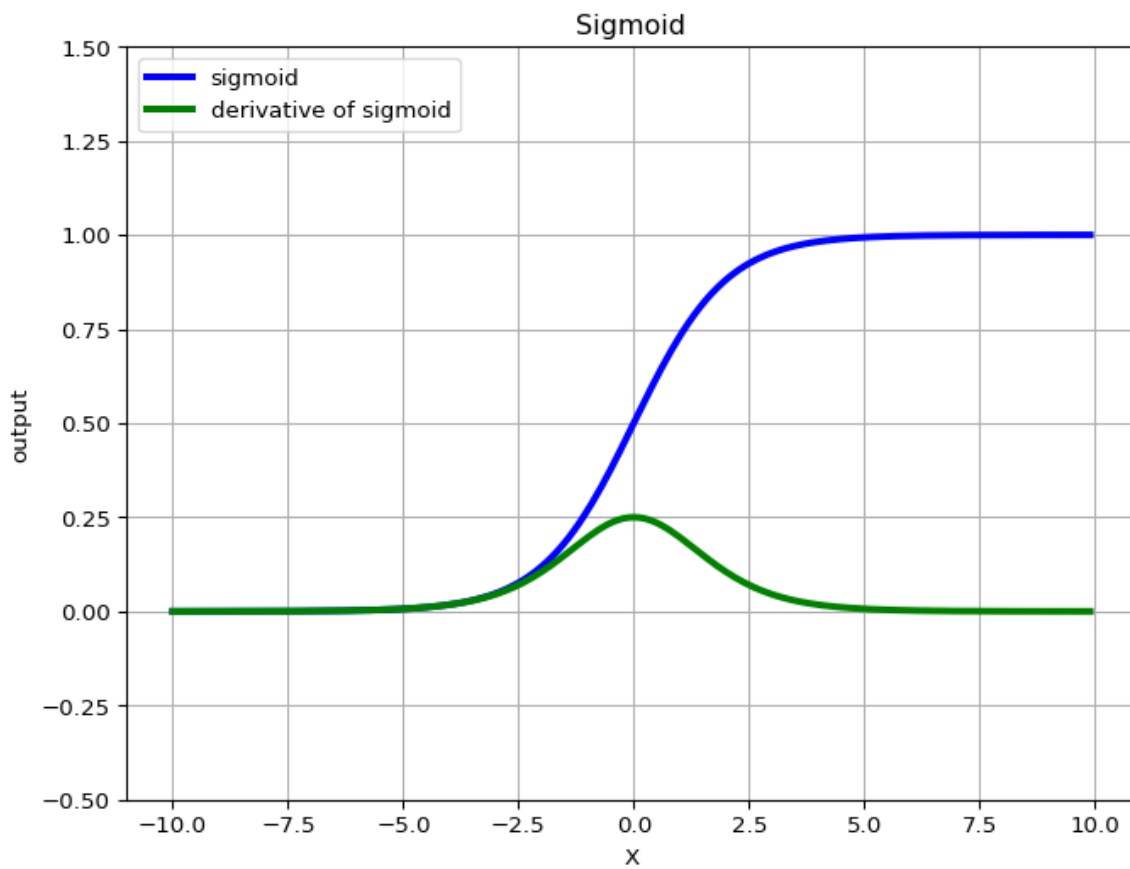


Figure 2: The sigmoid function and its derivative.

2.2 Hyperbolic tangent

The hyperbolic tangent, or tanh is sometimes called the bipolar sigmoidal function. It maps real numbers into the interval $(-1, 1)$, having horizontal asymptotes at $y = \pm 1$.

The advantage of tanh over sigmoid is that the negative inputs will be mapped strongly negative and the zero inputs will be mapped near zero in the tanh graph, and the graph of $\tanh(x)$ is symmetric with respect to the origin.

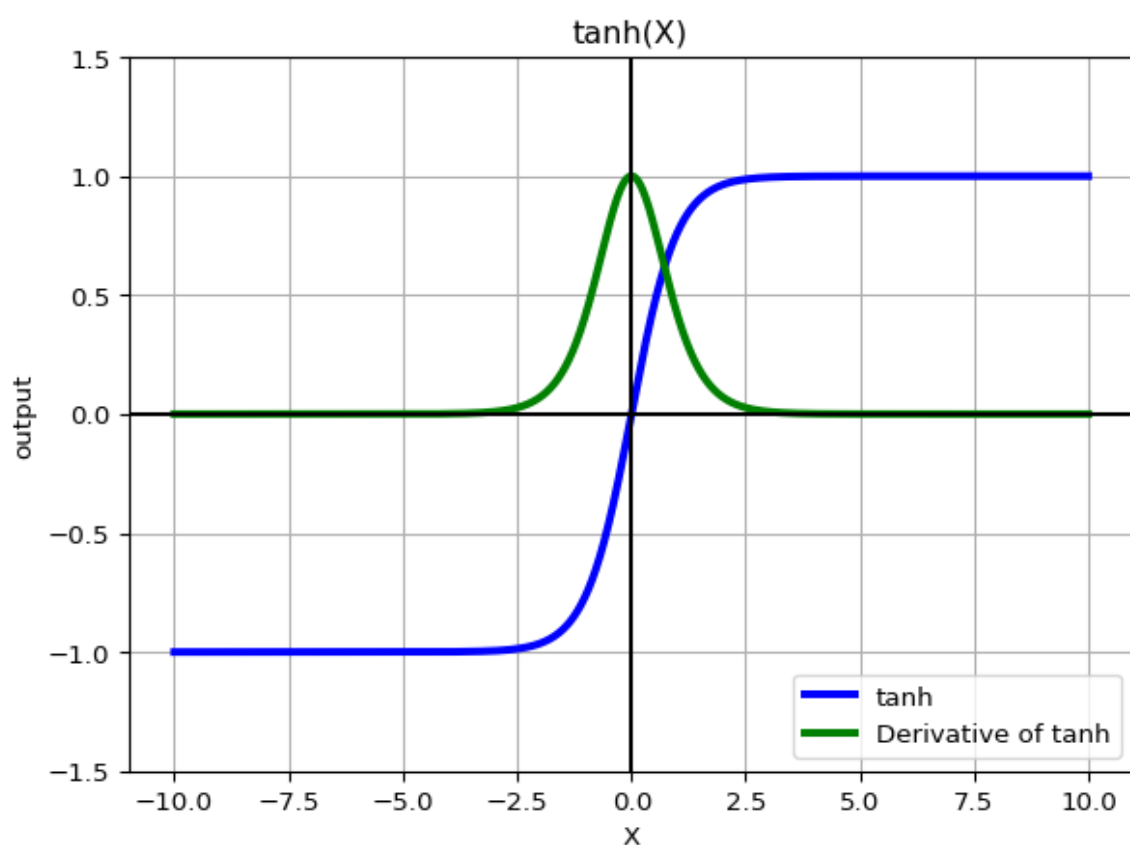


Figure 3: The tanh function and its derivative.

2.3 Rectified Linear Unit (ReLU)

A rectified linear unit (ReLU) is easy to optimize because it is very close to linear. The only difference between a linear unit and a ReLU is that the latter outputs zero across half of its domain.

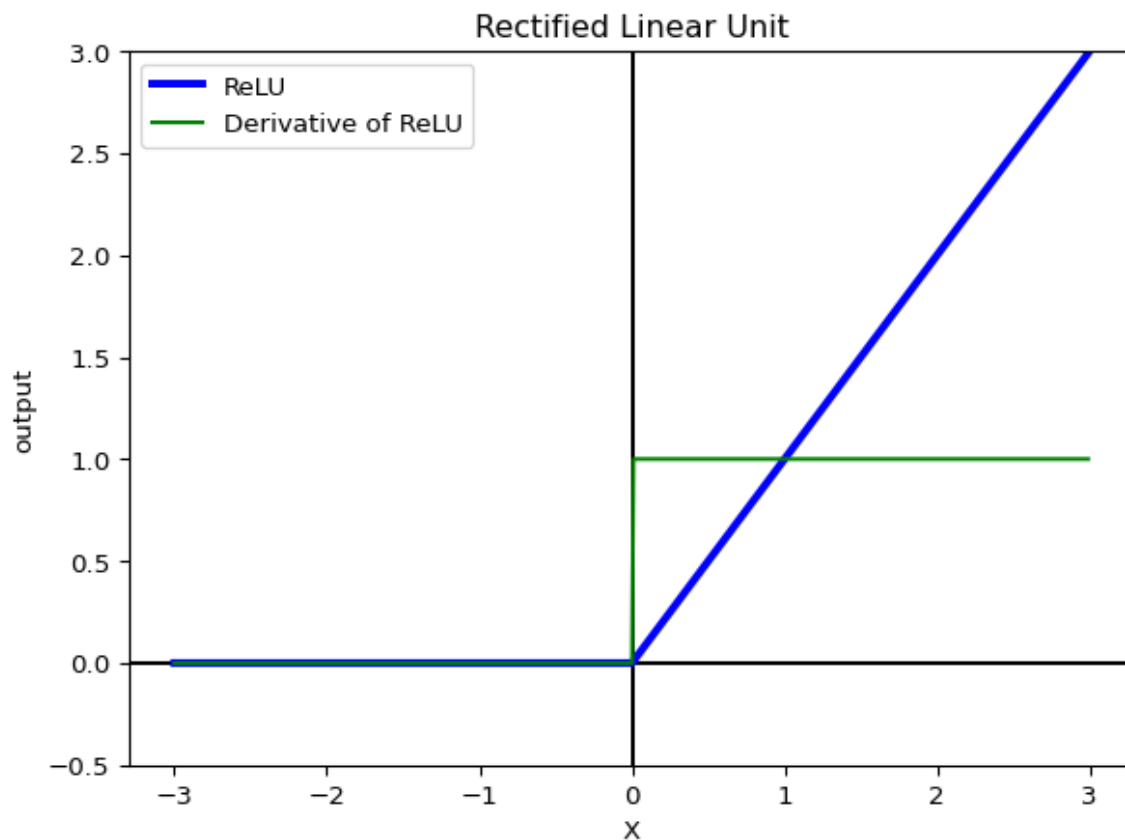


Figure 4: The ReLU function and its derivative.

3 Other important factors to consider

The MLP section in Dive Into Deep Learning introduces some additional aspects of model training that are important to consider. These include:

- How to [initialize](#) the model, particularly with respect to the problems of vanishing and exploding gradients
- More on how to improve the generalizability of the model, including using techniques such as [early stopping](#), as well as the concept of [dropout](#), another regularization technique.

Please [read and understand the sections linked above](#), and we will examine them in more detail when we get to the next section on CNNs.

